XGBOOST

how it works, and what every setting does

A reference, not a tutorial. Covers every setting xgboost has, including the ones this pipeline does not use yet.

The worked example at the end was measured on the real train_xgboost v7 model. None of those numbers are invented.

Nifty 50 signal pipeline

2026-08-05

1. WHAT A DECISION TREE IS

A tree is a stack of yes/no questions ending in an answer.

```
        is volatility  < 2.0 ?
           yes -> is trend < 5.0 ?          no  -> is volume < 100 ?
                    yes -> answer 0.13               yes -> answer -0.02
                    no  -> answer 0.41               no  -> answer  0.08
```

A row walks down, answering each question, until it lands in a LEAF.
The leaf holds a number. That number is the tree's answer for that row.

The words you need

NODE	one question
SPLIT	the act of turning a node into two children
LEAF	the bottom -- no more questions, just a number
DEPTH	how many questions from the top to the deepest leaf
COVER	the total weight of rows that reached a node

Why a tree and not a formula

A formula like score = 2*volatility + 3*trend treats every row the same.
A tree can say 'in a calm market look at trend, in a wild one look at volume'.
That is why trees beat linear models on messy data like markets.

The weakness of ONE tree

A single tree either stays small and misses a lot, or grows huge and memorises
the training data. Neither is useful. Boosting is the fix.

2. WHY MANY TREES -- BOOSTING

One tree guesses. The next fixes its mistakes.

```
   tree 1   guesses roughly. gets a lot wrong.
   tree 2   is trained on WHAT TREE 1 GOT WRONG. fixes some of it.
   tree 3   is trained on what 1+2 still get wrong.
   ...
   tree 500 is fixing the last small errors.

Each tree is trained on the LEFTOVER ERROR of everything before it.
The prediction is the SUM of all the trees.
```

This is different from a Random Forest

```
   RANDOM FOREST   trees are built INDEPENDENTLY and averaged.
                   each tree tries to solve the whole problem alone.
                   so each tree must be DEEP and strong.

   BOOSTING        trees are built IN ORDER, each correcting the last.
                   each tree only has to be slightly better than nothing.
                   so each tree can be SHALLOW.

That is why depth 6 is normal for boosting and terrible for a forest.
A forest of depth-6 trees is 300 copies of the same weak stump.
```

learning_rate is the volume knob

```
        prediction = tree1*lr + tree2*lr + tree3*lr + ...

lr = 1.0   each tree contributes fully. fast, and it overshoots.
lr = 0.05  each tree contributes 5%. slow, careful, needs many more trees.

The rule: halve the learning rate, roughly double the number of trees.
```

3. HOW XGBOOST CHOOSES A SPLIT

It tries every feature, every cut point, and keeps the best.

```
For one node holding some rows, it asks:

   for each feature:
       for each candidate cut value:
           how much would splitting here IMPROVE things?
   keep the single best.  if even the best is bad, do not split.
```

What 'improve' means -- the gain

```
        gain  =  score(left) + score(right)  -  score(if we do not split)  -  gamma
```

```
score() is a measure of how well a group of rows is explained.
Splitting helps if the two halves TOGETHER explain more than the whole did.

gamma is subtracted from every gain. It is a toll you must pay to split.
If gain comes out negative, the split is refused and the node becomes a leaf.
```

What a leaf value is

```
        leaf value  =  -(sum of gradients) / (sum of hessians + reg_lambda)
```

```
In plain words: the average error of the rows in that leaf, damped by lambda.
Bigger reg_lambda -> smaller leaf values -> a more cautious model.
```

Where each setting acts on this one formula

```
   max_bin           how many cut values are even tried
   colsample_*       which features are offered to try
   gamma             subtracted from the gain -- a toll per split
   min_child_weight  both sides must have enough COVER, or refuse
   reg_lambda        in the denominator -- shrinks every leaf value
   reg_alpha         pushes small leaf values to exactly zero
   max_delta_step    a hard cap on how big a leaf value may be

Six settings, one formula. That is why they interact.
```

4. THE FOUR GROUPS

Every setting either GIVES the model room, or TAKES room away.

ROOM TO FIT	BRAKES
more trees	low learning rate
deeper / more leaves	gamma
more bins	min_child_weight
	reg_lambda, reg_alpha
	subsample, colsample_*
	early stopping

Overfitting = room won. Underfitting = brakes won.

Group 1 -- HOW BIG each tree is

grow_policy, max_depth, max_leaves

Group 2 -- HOW MANY trees, and how fast they learn

n_estimators, learning_rate, early_stopping_rounds

Group 3 -- WHAT each split is allowed to look at

subsample, colsample_bytree, colsample_bylevel, colsample_bynode, max_bin

Group 4 -- WHEN a split is refused

gamma, min_child_weight, reg_lambda, reg_alpha, max_delta_step

The order to tune them

1. get the SIZE roughly right (group 1)
2. set learning_rate low and n_estimators high, with early stopping (group 2)
3. add sampling for variance (group 3)
4. only then reach for the regularisers (group 4)

Most people do it backwards and fight group 1 with group 4.

5. GROUP 1 -- THE SIZE OF EACH TREE

grow_policy decides WHICH size knob is real. Set this first.

```
depthwise   grow level by level. all leaves at depth 1, then all at 2 ...
            MAX_DEPTH RULES.  max_leaves is ignored.
            predictable, balanced trees. the classic setting.

lossguide   grow wherever the loss drops most, ignoring balance.
            MAX_LEAVES RULES.  max_depth only caps how long a branch gets.
            lopsided trees that spend their budget where the error is.
            like LightGBM. usually stronger, and easier to overfit.
```

max_depth

```
what   the longest chain of questions allowed
why    depth d gives up to 2^d leaves. depth 10 = 1,024. depth 20 = 1,048,576.
       it doubles every level, so small changes are not small.
values 3-6  shallow, strong brake, classic boosting
       6-10 normal
       >10  usually memorising unless the data is huge
       0    unlimited (safe under lossguide, dangerous under depthwise)
```

max_leaves

```
what   the total number of leaves allowed in one tree
why    under lossguide this IS the capacity knob. it is a direct budget:
       64 leaves means 64 different answers, however they are arranged.
values 31 is LightGBM's default and a sane start
       64-256 for bigger data
       0 = unlimited
```

Which should you use

```
Pick lossguide + max_leaves if you want power and will regularise properly.
Pick depthwise + max_depth if you want predictable, explainable trees.
Do not set both hoping they combine -- only one of them is live at a time.
```

6. GROUP 2 -- HOW MANY TREES, HOW FAST

n_estimators and learning_rate are ONE decision, not two.

```
   lr 0.3  + 100 trees      fast, rough, easy to overshoot
   lr 0.1  + 300 trees      common default
   lr 0.05 + 1000 trees     careful
   lr 0.01 + 5000 trees     very careful, very slow
```

All four can reach a similar fit. Lower lr generalises slightly better
because each tree is a smaller, more cautious step.

early_stopping_rounds -- the most under-used setting

```
   what   watch a validation set. if it has not improved for N rounds, STOP.
   why    you no longer have to guess n_estimators. set it high and let the
          data decide. this is the single best defence against overfitting
          and against wasting hours.
   values 20-50 typical. it needs an eval_set at fit time or it cannot work.
```

WHEN IT IS ON, n_estimators STOPS BEING A COUNT AND BECOMES A CEILING.

```
   without early stopping:  n_estimators 6000 -> exactly 6000 trees, always
   with early stopping:     n_estimators 6000 -> maybe 400, maybe 6000
```

What it costs you if it is off

Every round past the best one is doing one of two things: memorising the
training data, or adding empty trees that contribute nothing. Both cost
you wall-clock, disk, and prediction time.

7. GROUP 3 -- WHAT EACH SPLIT CAN SEE

The idea: hide things from the model so it cannot lean on one crutch.

subsample

```
what   the fraction of ROWS each tree sees. 0.8 = 80%, redrawn per tree.
why    two trees seeing different rows learn different things, and the
       differences cancel out as noise. also faster.
values 0.5-1.0. below 0.5 usually starves the trees.
```

the colsample family -- three knobs, same idea, different timing

```
colsample_bytree    pick features ONCE per tree
colsample_bylevel   RE-pick at each depth level
colsample_bynode    RE-pick at EVERY split          <- strongest

Think of it as how often you reshuffle the deck:
   bytree  = deal one hand, play the whole tree with it
   bylevel = new hand each level
   bynode  = new hand every single question
```

WHICH IS BETTER

bynode, in almost every case, and more so the deeper the tree.

With bytree, a strong feature that survives the draw can be used at every level all the way down -- the tree leans on it and never learns the alternatives. With bynode it is absent from most splits, so the model is forced to find other ways to say the same thing. That is what makes the trees genuinely different from each other, which is the whole point.

THEY MULTIPLY -- the trap

```
       effective  =  bytree  x  bylevel  x  bynode

0.5 x 0.5 x 0.5  =  0.125     only 12.5% of features per split
```

Set ONE of them and leave the others at 1.0. Otherwise you cannot reason about the number, and neither can anyone reading your config.

max_bin

```
what   feature values are bucketed into this many cut points to try.
why    fewer bins = fewer candidate splits = faster and more regularised.
       more bins = finer thresholds = more capacity to fit and to overfit.
values 256 default. 128 faster. 512 finer. only matters with tree_method=hist.
```

8. GROUP 4 -- WHEN A SPLIT IS REFUSED

These four are the brakes. They are also the easiest to over-apply.

gamma (also called min_split_loss)

```
what   the minimum gain a split must produce, or it is refused outright.
why    a toll. stops the model making splits that barely help, which are
       usually fitting noise.
values 0 default. 0.1-1.0 is a real brake. above ~5 most splits die.
note   this is the harshest brake -- it says NO, not 'smaller'.
```

min_child_weight

```
what   the minimum COVER (sum of weights) each side of a split must hold.
why    stops tiny leaves that fit a handful of odd rows.
values 1 default. higher = more conservative.
TRAP   it counts WEIGHT, not ROWS. see the next page.
```

reg_lambda (L2)

```
what   sits in the denominator of the leaf value. shrinks every leaf.
why    smooth, gentle damping. nothing goes to zero, everything gets smaller.
values 1 default. 0.1 nearly off. 10-100 strong.
```

reg_alpha (L1)

```
what   pushes small leaf values to EXACTLY zero.
why    it prunes. leaves that were barely contributing disappear entirely.
values 0 default. above ~1 it starts removing real structure.
```

L1 vs L2 in one line

```
L2 makes everything smaller.   L1 makes small things vanish.
```

max_delta_step

```
what   a hard cap on how far one leaf value may move in a round.
why    protects against one tiny group of heavily-weighted rows taking a
       single violent step. rarely needed -- EXCEPT with class weights.
values 0 = off. 1-10 when using strong weights on rare classes.
```

9. sample_weight -- AND HOW IT CHANGES EVERYTHING

sample_weight says how much each ROW counts.

```
        weight 2  =  this row pulls twice as hard as a weight-1 row
                     exactly as if you had copied it -- without copying it
```

Two common reasons to use it:

```
   CLASS BALANCE   rare classes get bigger weights so they are not ignored
   CONFIDENCE      rows you trust more get bigger weights
```

Why it is not just another setting

COVER -- the number min_child_weight compares against -- is the SUM of sample_weight, not a row count. So changing the weights silently changes what min_child_weight means, without touching min_child_weight.

Worked example

```
   min_child_weight = 5

   all weights = 1     -> a leaf needs 5 rows
   rare class weight 20 -> a leaf needs 0.25 rows, i.e. ONE row is enough
   common class weight 0.2 -> a leaf needs 25 rows
```

The brake is now much tighter on the common class than the rare one.

THE CONTRADICTION

min_child_weight exists to stop tiny leaves fitting noise. But rare classes have the FEWEST rows and are the easiest to overfit -- and inverse-frequency weighting gives them the BIGGEST weights, which removes their brake.

So the protection lands on the class that needed it least.

This is not an xgboost bug. It is what happens when you weight by rarity and then cap by weight. Be aware of it; there is no setting that fixes it, because xgboost has no row-count floor. The options are: lower the weights, use max_delta_step instead, or accept that the knob only controls the common class.

10. WHO SUPERSEDES WHOM

Seven rules. Learn these and the config stops surprising you.

```
1. grow_policy decides which size knob exists.
      depthwise -> max_leaves is IGNORED
      lossguide -> max_depth only caps branch length; max_leaves rules

2. early_stopping_rounds beats n_estimators.
      when on, n_estimators is a ceiling, not a count.
      when off, every round is built no matter how bad it gets.

3. gamma and min_child_weight are AND, not OR.
      a split must pass BOTH. the stricter one wins every time.
      lowering one does nothing if the other is what is blocking.

4. the colsample knobs MULTIPLY.
      bytree x bylevel x bynode. set one, leave the rest at 1.0.

5. sample_weight redefines min_child_weight.
      the weights are set outside the config, and they change what the
      number means. always read them together.

6. max_delta_step caps what learning_rate can do.
      lr scales every update; max_delta_step puts a ceiling on any single one.

7. reg_alpha can undo max_leaves.
      you may allow 256 leaves and L1 prunes half of them to zero. the tree
      you get is smaller than the one you asked for -- and if L1 and gamma
      block the very first split, you get a single-leaf tree that does nothing.
```

The general shape of it

```
   Group 1 grants capacity.  Group 4 takes it away.
   Group 4 always wins, because it acts at every single split.

So a config can say 'unlimited' and still produce stumps. Always check what
the model ACTUALLY built, not what you asked for.
```

11. RECIPES

Overfitting -- great on train, bad on test

1. turn early stopping on   (free, do this first)
2. lower learning_rate, raise n_estimators
3. reduce max_depth / max_leaves
4. subsample 0.8, colsample_bynode 0.8
5. raise min_child_weight
6. raise reg_lambda
7. gamma LAST -- it is the harshest and the easiest to overdo

Underfitting -- bad on both

1. is a brake too tight? check gamma and reg_alpha FIRST
2. raise max_depth / max_leaves
3. raise learning_rate, or add trees
4. raise max_bin

Too slow

1. early stopping (usually the entire problem)
2. lower max_bin to 128
3. lower n_estimators and raise learning_rate
4. subsample 0.7
5. tree_method='hist' -- should already be on

Rare classes being ignored

1. sample_weight or class weights
2. then RE-CHECK min_child_weight -- the weights just changed what it means
3. max_delta_step 1-5 to stop violent updates on the weighted rows
4. judge with per-class recall and PR-AUC, never accuracy

The one check people skip

After any run, look at the trees you actually got: how many, how many leaves,
how many are single-leaf stumps. If it does not match what you configured,
something in group 4 is overruling group 1.

12. MULTICLASS, MISSING VALUES, AND THE REST

7 classes means 7 trees per round

```
objective   multi:softprob  -> a probability for each class
num_class   7

n_estimators 6000  ->  6000 x 7  =  42,000 trees actually built
```

Each round builds one tree PER CLASS. Each tree scores its own class.
The class with the highest total score wins. So '6000 trees' in the config
is 42,000 trees in the file -- worth knowing before you size anything.

Missing values are handled natively

At every split xgboost learns a DEFAULT DIRECTION for NaN: it tries sending
them left, then right, and keeps whichever scored better. So NaN becomes a
branch of its own, learned from the data.

This is why you must not fill NaN with 0 or the mean. 'No value' is
information, and filling it destroys that information.

Settings not yet mentioned

```
tree_method        'hist' -- bucketed, fast, and what makes NaN handling work
scale_pos_weight   TWO-CLASS ONLY. does nothing on multiclass.
monotone_constraints
                   force a feature to only push one direction. use when you
                   KNOW a relationship, e.g. more risk must never mean more BUY.
interaction_constraints
                   forbid two features from appearing in the same branch.
base_score         the starting guess before any tree. rarely touched.
n_jobs             cores. -1 = all of them.
random_state       the seed. changes sampling, so it changes the model.
```

The two that sound useful and are not

```
scale_pos_weight on multiclass -- silently does nothing
colsample_bytree alongside bynode -- multiplies, hides the real number
```

13. EVERY SETTING XGBOOST HAS -- INCLUDING THE ONES WE DO NOT US

xgboost 3.3 exposes 40 settings. You currently touch 22. Here are the rest.

Worth trying on this problem

```
colsample_bylevel   re-pick features once per DEPTH LEVEL. sits between bytree
                    and bynode. 0.7 is a gentle middle ground.

monotone_constraints
                    force a feature to only ever push one way. a tuple, one entry
                    per feature: 1 = must increase, -1 = must decrease, 0 = free.
                    USE IT when you KNOW a relationship and the model keeps
                    inventing the opposite in some corner of the data.
                    e.g. higher stress must never increase a BUY score.
                    it costs a little accuracy and buys a model you can defend.

interaction_constraints
                    a list of groups; features may only be combined WITHIN a group.
                    stops the model building 47-way interactions that cannot
                    possibly generalise. strong medicine against overfitting on
                    wide feature sets -- worth trying with 400+ features.

num_parallel_tree   build N trees per round and average them -- boosted forest.
                    1 = plain boosting. 3-10 adds variance reduction per round.
                    multiplies your tree count, so watch the wall-clock.

sampling_method     'uniform' (default) or 'gradient_based'. the second samples
                    rows in proportion to how wrong they currently are, so you can
                    subsample far more aggressively (0.1) with little loss.
                    GPU only in most builds -- check before relying on it.

max_cat_to_onehot / max_cat_threshold / enable_categorical
                    native categorical handling instead of your integer encoding.
                    would let xgboost treat bucket labels as real categories rather
                    than numbers with a false order. a genuine experiment for you.

feature_weights     make some features more likely to be sampled by colsample.
                    a soft way to express a prior about which features matter.
```

Set once and forget

```
booster             'gbtree' default. 'dart' drops trees during training (a form
                    of regularisation, slower). 'gblinear' is not a tree model.
base_score          the starting guess before any tree. leave it alone.
missing             which value counts as missing. default NaN, which is right.
device              'cpu' or 'cuda'.
validate_parameters warn about unknown parameters. useful while experimenting.
importance_type     which importance get_score returns. SHAP is better anyway.
```

13. EVERY SETTING XGBOOST HAS -- INCLUDING THE ONES WE DO NOT US

Does nothing for you

```
scale_pos_weight    TWO-CLASS only. silently ignored on multiclass.
multi_strategy      'one_output_per_tree' (default) or 'multi_output_tree'.
                    the second builds ONE tree predicting all 7 classes at once --
                    far fewer trees, usually a bit weaker. worth one experiment.
```

14. OBJECTIVES AND EVAL METRICS

The objective is WHAT the model is trying to minimise. It is not a setting you tune -- it defines the problem.

```
multi:softprob   7 probabilities per row. what you use. right for 7 classes.
multi:softmax    the class only, no probabilities. never better -- you lose
                 the confidence numbers and gain nothing.
binary:logistic  TWO classes only. using it on 7 is a real and common mistake.
rank:pairwise    learn an ORDER rather than a class. worth thinking about if
                 what you actually want is 'which minutes are most worth
                 trading' rather than 'which class is this minute'.
```

eval_metric is what early stopping WATCHES. It changes when training stops.

```
mlogloss     multiclass log loss. what you use. punishes confident mistakes
             hardest, and it is smooth, which is what early stopping needs.
merror       plain error rate. ignores confidence. a model that is 51% sure
             and one that is 99% sure score identically. rarely the right choice.
auc / aucpr  ranking quality. aucpr is far better on imbalanced data because
             it ignores the true negatives -- and 74% of your rows are NO_TRADE.
             worth trying as a second metric.
```

You can pass a LIST

```
eval_metric = ['mlogloss', 'merror']
```

The LAST one is what early stopping uses. The others are logged only. That is a cheap way to watch a metric you care about without letting it control the run.

The thing to remember

Accuracy on a dataset that is 74% one class is close to meaningless -- predicting that class every time already scores 74%. Judge with per-class recall and PR-AUC. The objective and eval_metric should reflect the decision you actually care about, not the one that is easiest to compute.

15. A WORKED EXAMPLE -- A REAL MODEL

Configured versus what came out. Measured, not guessed.

```
   asked for:   n_estimators 6000
                max_depth 0        (unlimited)
                max_leaves 0       (unlimited)
                grow_policy lossguide

   got:         42,000 trees      (6000 x 7 classes)
                119 leaves per tree on average
                6,648 leaves in the largest
                8,801 trees are a SINGLE LEAF -- 21% do nothing
```

Why unlimited did not mean huge

```
Nothing in group 1 was limiting. These, from group 4, were:

   gamma 0.3             every split must beat a 0.3 toll
   min_child_weight 5.0  both sides need cover >= 5
   reg_alpha 1.0         L1, pruning weak leaves to zero

When all three refuse the first split, the tree is one leaf. 8,801 times.
```

A real split from tree 0

```
   node   question                              gain      cover
   0-0    candle_pattern_vote_sum   < 0.0     9250.3    74290.2
   0-1      structural_position_120m < 5.0    1286.7    40712.4
   0-2      structural_position_60m  < 2.0     917.5    33577.8

Cover 74,290 at the root is the SUM OF WEIGHTS of all training rows,
not the row count. Leaf values in this model run from -0.077 to +0.179 --
small, because learning_rate 0.05 keeps every step small.
```

The lesson

```
The config was internally inconsistent: one group granted unlimited capacity,
another removed it, and a fifth of the model came out empty. Every setting
did exactly what it says. The combination was never checked.
```